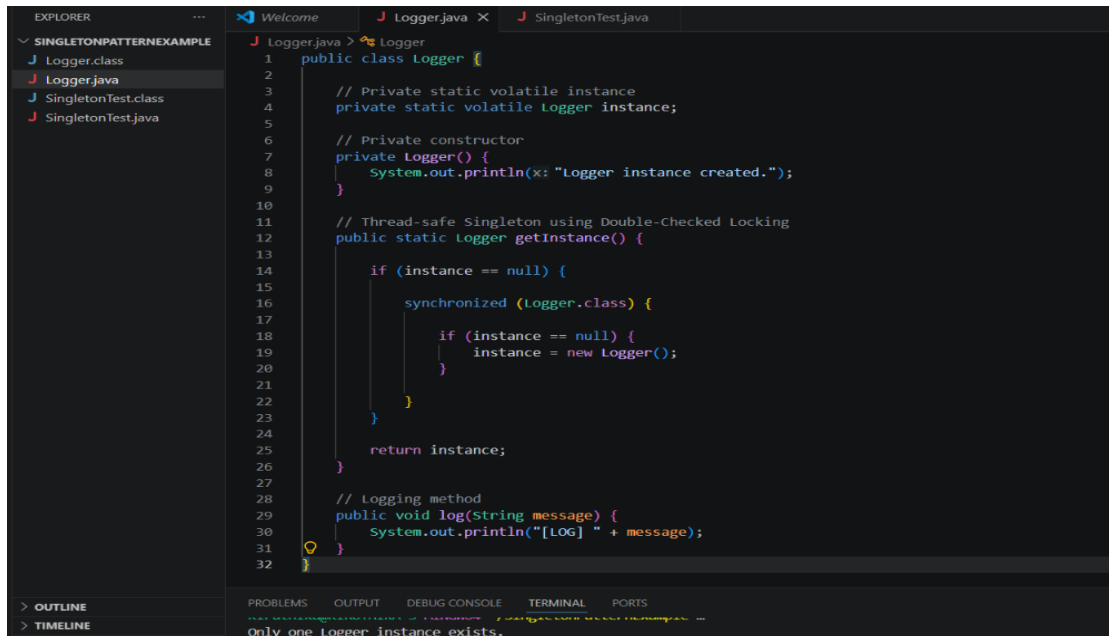


DIGITAL NURTURE 5.0 DEEPSKILLING

Week 1 – HANDS-ON

Design principles & Patterns

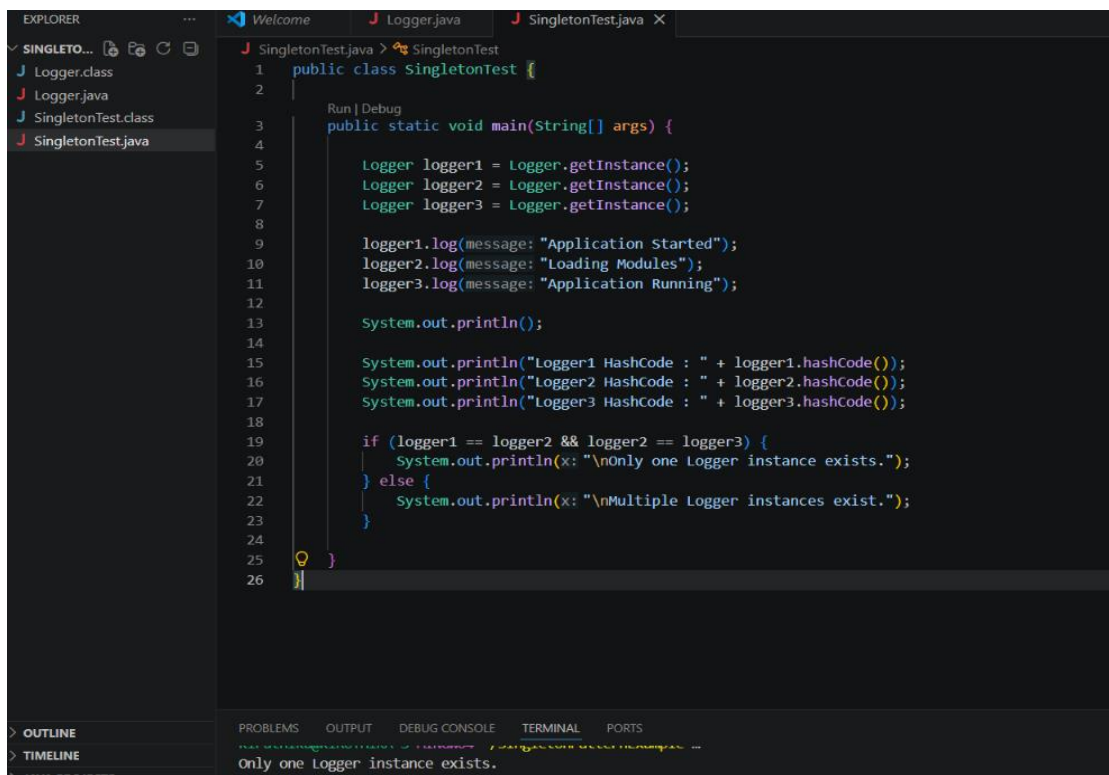
Exercise 1: Implementing the Singleton Pattern



The screenshot shows an IDE with the Explorer pane on the left displaying a project named 'SINGLETONPATTERNEXAMPLE' containing files 'Logger.class', 'Logger.java', 'SingletonTest.class', and 'SingletonTest.java'. The main editor displays 'Logger.java' with the following code:

```
1 public class Logger {
2
3     // Private static volatile instance
4     private static volatile Logger instance;
5
6     // Private constructor
7     private Logger() {
8         System.out.println(x: "Logger instance created.");
9     }
10
11     // Thread-safe Singleton using Double-Checked Locking
12     public static Logger getInstance() {
13
14         if (instance == null) {
15             synchronized (Logger.class) {
16                 if (instance == null) {
17                     instance = new Logger();
18                 }
19             }
20         }
21
22         return instance;
23     }
24
25     // Logging method
26     public void log(String message) {
27         System.out.println("[LOG] " + message);
28     }
29 }
```

The bottom panel shows the 'PROBLEMS' tab with the message: 'Only one Logger instance exists.'



The screenshot shows the same IDE with 'SingletonTest.java' open in the main editor. The Explorer pane on the left shows the project structure. The code in 'SingletonTest.java' is as follows:

```
1 public class SingletonTest {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         Logger logger1 = Logger.getInstance();
7         Logger logger2 = Logger.getInstance();
8         Logger logger3 = Logger.getInstance();
9
10        logger1.log(message: "Application Started");
11        logger2.log(message: "Loading Modules");
12        logger3.log(message: "Application Running");
13
14        System.out.println();
15
16        System.out.println("Logger1 hashCode : " + logger1.hashCode());
17        System.out.println("Logger2 hashCode : " + logger2.hashCode());
18        System.out.println("Logger3 hashCode : " + logger3.hashCode());
19
20        if (logger1 == logger2 && logger2 == logger3) {
21            System.out.println(x: "\nOnly one Logger instance exists.");
22        } else {
23            System.out.println(x: "\nMultiple Logger instances exist.");
24        }
25    }
26 }
```

The bottom panel shows the 'PROBLEMS' tab with the message: 'Only one Logger instance exists.'

```
1 public class SingletonTest {
2
3     Run | Debug
    public static void main(String[] args) {
4
5         Logger logger1 = Logger.getInstance();
6         Logger logger2 = Logger.getInstance();
7         Logger logger3 = Logger.getInstance();
8
9         logger1.log(message: "Application Started");
10        logger2.log(message: "Loading Modules");
    }
```

Kiruthika@KIRUTHIKA-S MINGW64 ~/SingletonPatternExample
\$ javac Logger.java SingletonTest.java
• java SingletonTest
Logger instance created.
[LOG] Application Started
[LOG] Loading Modules
[LOG] Application Running

Logger1 HashCode : 1933863327
Logger2 HashCode : 1933863327
Logger3 HashCode : 1933863327

Only one Logger instance exists.
Kiruthika@KIRUTHIKA-S MINGW64 ~/SingletonPatternExample
o \$

Exercise 2 : Implementing the Factory Method Pattern

```
1 interface Document {
2     void open();
3 }
```

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• \$ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• \$ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
o \$

The screenshot shows an IDE with the Explorer panel on the left displaying a project structure for a Factory Method Pattern example. The main editor displays the `DocumentFactory.java` file, which defines an abstract class `DocumentFactory` with an abstract method `createDocument()`. The terminal at the bottom shows the execution of the program, which successfully creates and opens Word, PDF, and Excel documents.

```
EXPLORER
  FACTORY...
    Document.class
    Document.java
    DocumentFactory.class
    DocumentFactory.java
    ExcelDocument.class
    ExcelDocument.java
    ExcelFactory.class
    ExcelFactory.java
    FactoryTest.class
    FactoryTest.java
    PdfDocument.class
    PdfDocument.java
    PdfFactory.class
    PdfFactory.java
    WordDocument.class
    WordDocument.java
    WordFactory.class
    WordFactory.java

DocumentFactory.java
1  abstract class DocumentFactory {
2
3      abstract Document createDocument();
4
5  }
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$
```

The screenshot shows the same IDE with the `ExcelDocument.java` file open in the main editor. This class implements the `Document` interface by extending `ExcelDocument` and overriding the `open()` method to print "Excel Document Opened". The terminal output is identical to the previous screenshot, showing the successful execution of the program.

```
EXPLORER
  FACTORY...
    Document.class
    Document.java
    DocumentFactory.class
    DocumentFactory.java
    ExcelDocument.class
    ExcelDocument.java
    ExcelFactory.class
    ExcelFactory.java
    FactoryTest.class
    FactoryTest.java
    PdfDocument.class
    PdfDocument.java
    PdfFactory.class
    PdfFactory.java
    WordDocument.class
    WordDocument.java
    WordFactory.class
    WordFactory.java

ExcelDocument.java
1  class ExcelDocument implements Document {
2
3      public void open() {
4          System.out.println(x: "Excel Document Opened");
5      }
6
7  }
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$
```

The screenshot shows an IDE with the Explorer view on the left displaying a project structure for a Factory Method Pattern example. The main editor displays the `ExcelFactory.java` file, which contains the following code:

```
1 class ExcelFactory extends DocumentFactory {  
2  
3     Document createDocument() {  
4         return new ExcelDocument();  
5     }  
6  
7 }
```

The terminal at the bottom shows the execution of the program:

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
○ $
```

The screenshot shows the same IDE with the `FactoryTest.java` file open in the main editor. The code is as follows:

```
1 public class FactoryTest {  
2     public static void main(String[] args) {  
3  
4         Document d1 = f1.createDocument();  
5         d1.open();  
6  
7         DocumentFactory f2 = new PdfFactory();  
8         Document d2 = f2.createDocument();  
9         d2.open();  
10  
11         DocumentFactory f3 = new ExcelFactory();  
12         Document d3 = f3.createDocument();  
13         d3.open();  
14     }  
15  
16  
17  
18  
19 }
```

The terminal output is identical to the previous screenshot, showing the successful compilation and execution of the program:

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
○ $
```

The screenshot shows an IDE with the Explorer on the left listing files like Document.class, DocumentFactory.class, ExcelDocument.class, and PdfDocument.class. The main editor displays PdfDocument.java with the following code:

```
1 class PdfDocument implements Document {
2
3     public void open() {
4         System.out.println("PDF Document Opened");
5     }
6
7 }
```

The bottom panel shows the Terminal output:

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• $ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• $ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
○ $
```

The screenshot shows the same IDE with PdfFactory.java selected in the Explorer. The main editor displays PdfFactory.java with the following code:

```
1 class PdfFactory extends DocumentFactory {
2
3     Document createDocument() {
4         return new PdfDocument();
5     }
6
7 }
```

The bottom panel shows the Terminal output:

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• $ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• $ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
○ $
```

The screenshot shows an IDE with the Explorer panel on the left listing various Java files. The main editor displays the `WordDocument.java` file, which implements the `Document` interface. The `open()` method prints "Word Document Opened". The terminal at the bottom shows the execution of `javac *.java` and `java FactoryTest`, resulting in the output "Word Document Opened", "PDF Document Opened", and "Excel Document Opened".

```
class WordDocument implements Document {  
    public void open() {  
        System.out.println(x: "Word Document Opened");  
    }  
}
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
$
```

The screenshot shows the same IDE with the `WordFactory.java` file selected in the Explorer panel. The main editor displays the `WordFactory` class, which extends `DocumentFactory` and implements the `createDocument()` method to return a new `WordDocument` object. The terminal output is identical to the previous screenshot, showing the successful compilation and execution of the factory test.

```
class WordFactory extends DocumentFactory {  
    Document createDocument() {  
        return new WordDocument();  
    }  
}
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
$
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
● $ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
● $ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened
❏
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
○ $
```

Data Structures and Algorithms

Exercise 2: E-commerce Platform Search Function